EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA

TECNOLÓGICO NACIONAL DE MÉXICO

Tecnológico Nacional de México

Campus Querétaro

E14-Responsable de calidad

Producto de la asignatura:
Ingeniería de Software

Desarrolla por:

Hernandez Perrusquia Jhoel
21140780

Asesor docente:

Villeda Maldonado Julio Alejandro

Fecha de entrega:

09 de marzo de 2026

Grupo:
6A

Evaluación de Modularidad

La calidad de un sistema de software se mide por su capacidad de ser mantenido y escalado sin errores en cadena. Tras revisar la arquitectura de capas (**Controller-Service-Repository-Entity**) y la distribución de los 6 componentes principales, se emite la siguiente evaluacion:

A. Evaluación de cohesión (Fuerza Interna)

El sistema presenta una **cohesión funcional alta**, lo cual es el nivel óptimo en ingeniería de software.

- **Especialización de servicios:** Cada componente identificado por el Analista tiene una responsabilidad única. Por ejemplo, el **AutentificaciónService** se limita exclusivamente a la validación de credenciales del DDA, mientras que el **ConstanciaService** encapsula únicamente la lógica de renderizado y validación de horas.
- **Encapsulamiento de datos:** Los Repositorios (como **CursoRepository** e **InscripciónRepository**) centralizan el acceso a la base de datos MySQL, evitando que la lógica de negocio se mezcle con sentencias SQL directas.

B. Evaluación de acoplamiento (Interdependencia)

Se observa un **acoplamiento de datos y de control moderado**, necesario para la operación, pero con áreas de vigilancia.

- **Flujo unidireccional:** El Diseñador ha establecido capas claras (Cliente → Lógica → Datos), lo que garantiza que los cambios en la interfaz web no afecten la estructura de la base de datos .
- **Dependencias funcionales críticas:** El Desarrollador identifica que el **InscripciónService** depende obligatoriamente del **CursoService** para validar cupos antes de persistir datos . Este acoplamiento es lógico, pero debe gestionarse mediante interfaces para evitar que un error en el módulo de cursos bloquee las inscripciones .

Identificación de Riesgos Arquitectónicos

Desde la perspectiva de Calidad, se han detectado tres puntos críticos que podrían comprometer la estabilidad del sistema si no se atienden antes de la implementación:

1. **Riesgo de "componente obeso" (Inscripción/Avance):**
    - **Hallazgo:** Actualmente, el componente de **inscripción** está asumiendo la lógica de unión al curso y también el registro de avance de horas .
    - **Impacto:** Esto crea una responsabilidad mal definida; si el código crece, será difícil depurar errores de progreso académico sin afectar el módulo de inscripciones iniciales.
2. **Punto unico de falla en la autenticación:**
    - **Hallazgo:** El sistema depende críticamente del módulo de autenticación para todas las funciones administrativas .

    - **Impacto:** Un fallo en la API de validación del instructor dejaría inoperantes los 5 módulos restantes, generando una denegación de servicio total para la gestión de constancias.
3. **Saturación por procesamiento pesado (PDF):**
    - **Hallazgo:** La generación de documentos PDF es una tarea síncrona que consume altos recursos de servidor.
    - **Impacto:** Si múltiples docentes generan constancias simultáneamente al final del periodo, el tiempo de respuesta del sistema podría degradarse significativamente .

Recomendaciones de mejora

Para mitigar los riesgos anteriores y elevar la madurez del software, se proponen las siguientes 4 recomendaciones obligatorias:

1. **Segregación de la lógica de progreso:** Extraer la funcionalidad de "Registro de Avance" del componente de inscripción y crear un **ProgresoService** independiente. Esto mejora la cohesión y permite escalar el seguimiento académico de forma aislada.
2. **Implementación de generación asíncrona (colas):** El componente de **generación de constancias PDF** debe evolucionar hacia un modelo de "trabajo en segundo plano". El sistema debe recibir la solicitud, encolarla y notificar al usuario cuando el archivo esté listo para descarga, protegiendo la CPU del servidor.
3. **Capa de caché para catálogos:** Los datos estáticos del componente de **administración de catálogos** (departamentos y plazas) no cambian con frecuencia . Almacenar estas listas en memoria caché reducirá el acoplamiento con la base de datos y acelerará la carga de formularios de registro .
4. **Middleware de Autorización Centralizado:** En lugar de que cada controlador dependa directamente del **AutentificaciónService** , se debe implementar un interceptor global que valide los permisos antes de llegar a la lógica de negocio, simplificando la estructura de los componentes funcionales